Nullcon HackIM 2023 Crypto - Curvy Decryptor

Nullcon HackIM 2023 Crypto - Curvy Decryptor writeup: cryptography, ECC, invalid_curve, CRT, ECDLP

EVENT	CATEGORY	TOPIC	PUBLISHED
nullcon hackim	crypto	ECC	2023-08-21

Challenge Description

Curvy Decryptor 473 points Alice has hidden 2 flags in this challenge. And even though she is willing to decrypt most ciphers, she has some basic saveguards against stealing flags. Please submit flag1 here. nc 52.59.124.14 10005

Source Files

curvy_decryptor.py (./curvy_decryptor.py)
ec.py (./ec.py)
utils.py (./utils.py)

Source Analysis

From `curvy_decryptor.py`

PYTHON
76 lines / static highlight

```python
#!/usr/bin/env python3
import os
import sys
import string
from Crypto.Util import number
from Crypto.Util.number import bytes_to_long, long_to_bytes
from Crypto.Cipher import AES
from binascii import hexlify

from ec import *
from utils import *
from secret import flag1, flag2

#P-256 parameters
p =
0xffffffff00000001000000000000000000000000ffffffffffffffffffffffff
f
a = -3
b =
0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
b
n =
0xffffffff00000000ffffffffffffffffbce6faada7179e84f3b9cac2fc63255
1
curve = EllipticCurve(p,a,b, order = n)
G = ECPoint(curve,
0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c29
6,
0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f
5)

d_a = bytes_to_long(os.urandom(32))
P_a = G * d_a

printable = [ord(char.encode()) for char in string.printable]
```

```
def encrypt(msg : bytes, pubkey : ECPoint):
    x = bytes_to_long(msg)
    y = modular_sqrt(x**3 + a*x + b, p)
    m = ECPoint(curve, x, y)
    d_b = number.getRandomRange(0,n)
    return (G * d_b, m + (pubkey * d_b))

def decrypt(B : ECPoint, c : ECPoint, d_a : int):
    if B.inf or c.inf: return b''
    return long_to_bytes((c - (B * d_a)).x)

def loop():
    print('I will decrypt anythin as long as it does not talk
about flags.')
    balance = 1024
    while True:
        print('B:', end = '')
        sys.stdout.flush()
        B_input = sys.stdin.buffer.readline().strip().decode()
        print('c:', end = '')
        sys.stdout.flush()
        c_input = sys.stdin.buffer.readline().strip().decode()
        B = ECPoint(curve, *[int(_) for _ in B_input.split(',')])
        c = ECPoint(curve, *[int(_) for _ in c_input.split(',')])
        msg = decrypt(B, c, d_a)
        if b'ENO' in msg:
            balance = -1
        else:
            balance -= 1 + len([c for c in msg if c in
printable])
        if balance >= 0:
            print(hexlify(msg))
            print('balance left: %d' % balance)
        else:
            print('You cannot afford any more decryptions.')
            return
```

```
if __name__ == '__main__':
    print('My public key is:')
    print(P_a)
    print('Good luck decrypting this cipher.')
    B,c = encrypt(flag1, P_a)
    print(B)
    print(c)
    key = long_to_bytes((d_a >> (8*16)) ^ (d_a &
0xffffffffffffffffffffffffffffffff))
    enc = AES.new(key, AES.MODE_ECB)
    cipher = enc.encrypt(flag2)
    print(hexlify(cipher).decode())
    try:
        loop()
    except Exception as err:
        print(repr(err))
```

Curvy Decryptor part 1

The solution of Curvy Decryptor part 1 is to find out `flag1`

Curvy Decryptor part 2

The solution of Curvy Decryptor part 1 is to find out `flag2`

The `flag2` appears to be AES encrypted with a key which is

PLAIN TEXT

1 line / static highlight

```
key = long_to_bytes((d_a >> (8*16)) ^ (d_a &
0xffffffffffffffffffffffffffffffff))
```

The xor of most significant and least significant 64 bits of the 128 bits private key `d_a`

So in order to recover the `flag2` we will need to break the ECC and recover `d_a`

Analysing curvy_decryptor.py

PYTHON
10 lines / static highlight

```python
#P-256 parameters
p =
0xffffffff00000001000000000000000000000000ffffffffffffffffffffffff
f
a = -3
b =
0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
b
n =
0xffffffff00000000ffffffffffffffffbce6faada7179e84f3b9cac2fc63255
1
curve = EllipticCurve(p,a,b, order = n)
G = ECPoint(curve,
0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c29
6,
0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f
5)

d_a = bytes_to_long(os.urandom(32))
P_a = G * d_a
```

indeed look like a standard curve P-256 (note that a = -3 is equivalent to a = p - 3)

So there doesnt appear to be any standard weakness based on weak curve parameters.

The private key `d_a` is initialized to be per-instance system-random 128 bit number
And the Public key `P_a` is simply `G*d_a`

Encryption

PYTHON
6 lines / static highlight

```python
def encrypt(msg : bytes, pubkey : ECPoint):
    x = bytes_to_long(msg)
    y = modular_sqrt(x**3 + a*x + b, p)
    m = ECPoint(curve, x, y)
    d_b = number.getRandomRange(0,n)
    return (G * d_b, m + (pubkey * d_b))
```

The `msg` is encoded as the x coordinate of the point, the corresponding `y` is found so as to find the point on the curve to generate the point `m`
A secure random number `d_b` in range (0 - curve order) is generated as the nonce, The points `G*d_b` and `m + (pubkey * d_b)` are returned

Decryption

PYTHON
3 lines / static highlight

```python
def decrypt(B : ECPoint, c : ECPoint, d_a : int):
    if B.inf or c.inf: return b''
    return long_to_bytes((c - (B * d_a)).x)
```

It simply reverses the encrypt function if correct `d_a` is provided
i.e if we do

PYTHON

2 lines / static highlight

```
B,c = encrypt(flag1, P_a)
xx = decrypt(B,c)
```

We will get,

$$\text{decrypt}((G * d_b * d_a), \text{flag}_m + (P_a * d_b))$$

$$= (\text{flag}_m + (P_a * d_b) - G * d_b * d_a).x$$

$$= (\text{flag}_m + G * d_a * d_b - G * d_b * d_a).x$$

$$= (\text{flag}_m).x = \text{flag}$$

Main Loop

The main loop of the program just repeatedly asks for input of two EC points `B` and `c` and tries to decrypt it with the servers private key `d_a`
BUT
if the decryption contains `b'ENO'` i.e the start of the flag, it exits.
Otherwise it decreases the balance proportionate to the number of printable characters in the decryption.

So If we directly input the `B` and `c` corresponding to the `flag1`, it will abort and hence no flags for us :'(

But it doesnt care what points we ask it to decrypt.
So what if we try to decrypt the points `B`, `c + A`

We will get,

$$\text{decrypt}((G * d_b * d_a), \text{flag}_m + (P_a * d_b) + A)$$

$$= (\text{flag}_m + (P_a * d_b) + A - G * d_b * d_a).x$$

$$= (\text{flag}_m + A + G * d_a * d_b - G * d_b * d_a).x$$

$$= (\text{flag}_m + A).x$$

As long as we know A, we can always get the point from the x coordinate, and subtract A from it to get the original point from the curve for the sake of simplicity, we can even pick it to be `G`

or we can even try decrypting the points `B + A`, `c` which will lead to

$$\text{decrypt}((G * d_b * d_a) + A, \text{flag}_m + (P_a * d_b))$$

$$= (\text{flag}_m + (P_a * d_b) - (G * d_b + A) * d_a).x$$

$$= (\text{flag}_m + G * d_a * d_b - G * d_b * d_a - A * d_a).x$$

$$= (\text{flag}_m - A * d_a).x$$

if we choose `A` to be `G` or `-G`, we will end up with the point `flag - P_a` or `flag + P_a` and since we even know `P_a`, it will also work

With a high probability, we wont observe any `b'ENO'` in the resulting point, and if we do, we can always pick countless possibilities of `A` to make it work

PYTHON
45 lines / static highlight

```python
from ec import *
from utils import *
import pwn

p =
0xffffffff00000001000000000000000000000000ffffffffffffffffffffffff
f
a = -3
b =
0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
b
n =
0xffffffff00000000ffffffffffffffffbce6faada7179e84f3b9cac2fc63255
1
curve = EllipticCurve(p,a,b, order = n)
G = ECPoint(curve,
0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c29
6,
0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f
5)

HOST, PORT = "52.59.124.14", 10005
REM = pwn.remote(HOST, PORT)

REM.recvline() # My public key is:
pubkey = REM.recvline().strip()[6:-1].split(b',')
P_a = ECPoint(curve, int(pubkey[0]), int(pubkey[1]))

REM.recvline() # Good luck decrypting this cipher.
B_text = REM.recvline().strip()[6:-1].split(b',')
B = ECPoint(curve, int(B_text[0]), int(B_text[1]))
```

```
c_text = REM.recvline().strip()[6:-1].split(b',')
c = ECPoint(curve, int(c_text[0]), int(c_text[1]))

flag2_enc = bytes.fromhex(REM.recvline().strip().decode())

REM.recvline() # I will decrypt anythin as long as it does not
talk about flags.

def get_decryption(B,c):
    REM.sendline("{},{}".format(B.x, B.y))
    REM.sendline("{},{}".format(c.x, c.y))
    status = REM.recvline()
    if b'cannot afford' in status:
        return -1, None
    balance = int(REM.recvline().strip().split(b': ')[-1])
    return balance, bytes.fromhex(status.strip()[6:-1].decode())

bal, BG = get_decryption(B, c+G)
BG_int = int.from_bytes(BG)
y = modular_sqrt(BG_int**3+a*BG_int+b,p)  # getting the valid y
coordinate for the x
point_BG1 = ECPoint(curve, BG_int, y)
point_BG2 = -point_BG1
print(int.to_bytes((point_BG1-G).x, 32,'big'))
print(int.to_bytes((point_BG2-G).x, 32,'big'))
```

Note that we only get the x coordinate of the given point lifting the point would result in two points and we will need to try with both of them (x,y) and (x,-y)

The last part will also work with

PYTHON
7 lines / static highlight

```
bal, BG = get_decryption(B-G, c)
BG_int = int.from_bytes(BG)
y = modular_sqrt(BG_int**3+a*BG_int+b,p)  # getting the valid y
coordinate for the x
point_BG1 = ECPoint(curve, BG_int, y)
point_BG2 = -point_BG1
print(int.to_bytes((point_BG1-P_a).x, 32,'big'))
print(int.to_bytes((point_BG2-P_a).x, 32,'big'))
```

And we get the flag to part 1

`b'\x00\x00ENO{ElGam4l_1s_mult1pl1cativ3}'`

Analysing ec.py

PYTHON
66 lines / static highlight

```
from Crypto.Util.number import inverse

class EllipticCurve(object):
        def __init__(self, p, a, b, order = None):
                self.p = p
                self.a = a
                self.b = b
                self.n = order

        def __str__(self):
                return 'y^2 = x^3 + %dx + %d modulo %d' %
(self.a, self.b, self.p)

        def __eq__(self, other):
                return (self.a, self.b, self.p) == (other.a,
other.b, other.p)
```

```
class ECPoint(object):
        def __init__(self, curve, x, y, inf = False):
                self.x = x % curve.p
                self.y = y % curve.p
                self.curve = curve
                self.inf = inf
                if x == 0 and y == 0: self.inf = True

        def copy(self):
                return ECPoint(self.curve, self.x, self.y)

        def __neg__(self):
                return ECPoint(self.curve, self.x, -self.y,
self.inf)

        def __add__(self, point):
                p = self.curve.p
                if self.inf:
                        return point.copy()
                if point.inf:
                        return self.copy()
                if self.x == point.x and (self.y + point.y) % p
== 0:
                        return ECPoint(self.curve, 0, 0, True)
                if self.x == point.x:
                        lamb = (3*self.x**2 + self.curve.a) *
inverse(2 * self.y, p) % p
                else:
                        lamb = (point.y - self.y) *
inverse(point.x - self.x, p) % p
                x = (lamb**2 - self.x - point.x) % p
                y = (lamb * (self.x - x) - self.y) % p
                return ECPoint(self.curve,x,y)

        def __sub__(self, point):
                return self + (-point)

        def __str__(self):
```

```
                if self.inf: return 'Point(inf)'
                return 'Point(%d, %d)' % (self.x, self.y)

        def __mul__(self, k):
                k = int(k)
                base = self.copy()
                res = ECPoint(self.curve, 0,0,True)
                while k > 0:
                        if k & 1:
                                res = res + base
                        base = base + base
                        k >>= 1
                return res

        def __eq__(self, point):
                return (self.inf and point.inf) or (self.x ==
point.x and self.y == point.y)
```

Looking closely at the `ECPoint` class, one would note that on the initialization of the point with arbitrary `x, y` coordinates, it works as usual and doesnt check whether the supplied `x, y` satisfy the curve equation $y^2 = x^3 + ax + b \mod p$

This leads to an interesting vulnerability aka **Invalid Curve Attack**

Which can be noted by the facts that

1. The point addtion of two points P and Q over the curve $y^2 = x^3 + ax + b \mod p$if $P \neq Q$ is independent of both the curve parameters a and b
2. The point doubling i.e $P = Q$ is just dependent on P and Q and a but not on b again

This means that the group addition operation is independent of the parameter b, the number of points in the group of some point P is just dependent on P and a but independent of b

So if we choose a curve C' with parameter b' and pick a valid point P' on it, and run the point addition over the original curve C, the order of point P' when used on C will be the same as order of point P' when used on C'

This implies that we are not stuck with the original prime order of P-256, but we can vary $b = -3$ such that the order of the curve C' has small factors. We can then easily find points P' with those small factors as their order by using the fact that -

If G' is the generator of C' with order $o = f_1 f_2 f_3 ... f_n$, the point $G' * (o/f_1)$ will have the order f_1

Once we have sufficient number of small orders, we can take a chinese remainder theorem over them to recover the original private key `d_a`

To find the order of curve C' and the corresponding generators, we can utilize the greate library of sagemath

PYTHON
19 lines / static highlight

```python
def get_invalid_curves(a,b,n,cutoff=10**5):
    factors, total, i = {}, 0, 0
    while total < n*2:
        i += 1
        try:
            E = EllipticCurve(GF(p), [a,i])
            order = E.order()
            n_facs = order.factor()
        except ArithmeticError: #the parameter i defines a
```

```
singular curve
            continue
        for prime, power in n_facs:
            if prime > cutoff: # dont take any factors bigger
than it
                break
            gen = E.gen(0)*(order//prime)
            factors[prime] = [int(gen[0]), int(gen[1]),i]
            total *= prime
        print(i, total)
    # key : [gen_x, gen_y, b']
    return factors
```

And we can easily bruteforce a given point

PYTHON
4 lines / static highlight

```
def bruteforce(point, generator, order):
    for i in tqdm(range(order),desc=f"bruteforcing {order=}"):
        if point.y == (generator*i).y:
            return i
```

We get the following invalid curves and we just searched over 12 values of b!

PYTHON
58 lines / static highlight

```
invalid_curves = {
3:
[796922802392729808732453878318748234760976653650691635588175703862186575269 67,

1588565748715503091228803188812842712493681308085947237649466313079886711998 2, 12],
5:
[304635864562590527161741217247237884787973189397622915236519661 5
```

```
1233767925799,

145210266523356166306112195153902914110234006419489576808114067210
43438902186, 12],
7:
[74941609631667190224457894486700754683005392162205960445813610343
11676798234,

98511591492536111584332615786483658886888663084020823615343609813
895225923287, 0],
13:
[44238399751822344629155927349410921734336660036385908812849527496
419061724190,

11120913773080173377402108816240868388875386584846966540375366366
3899601005809, 3],
17:
[92741446879457843642919037071163126599639170318501218859760577847
93297477861,

86301980488975426887521079169756244075123784201450393961147458432
303011672794, 10],
19:
[10865725148883783971009589474386673905248688027125803361351041963
4191398226376,

85939059343162290921933874524377315775260886906764656684571310947
58391852209, 4],
37:
[68293383902662374822833102466651033088912283363194773184796445222
60556056309,

74701668267551028200304338837410580676774474001240882947774298643
661074111881, 3],
71:
[47689891150662520216418276050802771367044708366511766907543187521
601344242001,
```

```
769867235052584446118742354358874050185137585249217577650335402142
85112109625, 1],
97:
[1093524381327895976762698492711619330291159637003767830442148056
43475162939438,

4049419958213355139556010459259189644885441236899747036968572065845
5096277720, 3],
113:
[2475842305874220823820486444323131896857191883016682295763890607
9202832915346,

6468069421663339086501436202070790415033334005190480658080385826798
5332824228, 3],
179:
[8311563101649050482232865577789586416266078232566035967479206533
2260812135544,

3570714148691635381635812390035688867348826070958187562881962218726
1352405569, 4],
251:
[2258959779636525724629675812850577063879996176931082468786804101
0311103597978,

2273037584209940456012941256088249209399872401174958247300368287199
4848593450, 10],
389:
[6173741830680999690863059543783205227270026389202136141564002831
4169193468679,

3881262201290735870297109865298991093171093518058925249303983924446
4470874161, 12],
653:
[9694668034361392030009188060702797389146046409674180327317079721
9756170839615,

8284378616542571170972530821008092883966964624695007200260330024821
9042690214, 9],
```

```
823:
[10159107816987559575310999149490550696797813624096117238850927562132525316525 6,

10303457253039461887297603877250453486351456285884301634668455036581356870325, 1],
1151:
[4942947285962241518079147671001480777229821084370946279070977308149340420785,

16048516228456466259745940658231903287363041924322695524263376011778714624389, 7],
1229:
[9170924868892838157497030687995914325677991135597065911779823476 1550615769703,

85856020107303390216619790339131383987393998107943546966635616939179964220668, 1],
2447:
[1070910371096125709951362942133366829239137179860541790946439220 74841981090569,

382978477354463513466011867613359494649029744297276528251289886356822281005 45, 5],
4003:
[6963461236054763969297805073647558400000195034696325413489365933 1303767659709,

726769015467602170871118849779502791615579178439921393135585135151063916317 5, 6],
7103:
[1103235277408923562768338447688604495542910102082012557928250534 03232044044793,

34702628194678317337541067016532288256370093140368313590333192034888987762792, 7],
7489:
[6949761078970559517405873710624251310095013019092070246743103217
```

```
2354669590563,

430009897763776675209333288006757651500406045460376766981733820080
99239610730, 1],
13003:
[699943884313078560803225727319709172701510675110185176195305689148
12259046195,

516458890203756080543663359573520742571303209763412492240103439926
76177045239, 4],
16033:
[801508497707012807703792608028763322572456516072204368738417085695
83336291111,

674544430341446028077610394941799137322803885504551724865998781089
95578176702, 6],
19423:
[867633166961161462078462094430893760959665422819900718726987341242
75764832625,

149737328118884134208211291751940839166467399159371075622581631360
2284346637, 11],
30203:
[271403067691243642532128268899512507147829291806854555992846877025
13066987645,

274656683745400523587853269339045970479919040303785132976775162816
90992773738, 1],
52183:
[207868930062006681359805174813051989678715221307737005713272561802
24225598537,

384764507121596729890471198736739885960955160966481840672101031636
59625447149, 12],
72337:
[168646731360432786930401855723034857436771259992334199764373024710
94264721938,
```

```
3998211074784874058895788459831601080286548381466957694030470844
368796141014, 9],
81173:
[4196584713467566686308967062141269929720744625991527783293989960
5119013001686,

9759510259234686987574987361267652853497119825962442750768014877
098555985918, 8]
}
```

Attack procedure

1. For each b' and G' of order o'
2. Send $B = -G'$ and $c = G'$ we will get back x coordinate of $G' * (d_a + 1)$

> **Note that if we send 0, it will just return `b''` so no use**

3. Lift the x coordinate **Over C'** to get $G' * (d_a + 1)$ But this wont help us to figure out of the two possible points. For this
4. Send another $B = -G'$ and $c = G' * 2$ to get back x coordinate of $G' * (d_a + 2)$ To figure out the correct lifting of the x coordinate to get $G' * (d_a + 1)$
5. Bruteforce and recover the ECDLP to get $d'_a = (d_a + 1) \mod o'$
6. Combine all d'_a using chinese remainder theorem to get back original d_a
7. PROFIT ????

PYTHON
27 lines / static highlight

```
recovered_order = {}
for order, (gen_x, gen_y, b_i) in invalid_curves.items():
    gen = ECPoint(curve, gen_x, gen_y)
    bal, BG = get_decryption(-gen, gen) # gen*(da+1)
    BG_int = int.from_bytes(BG)
    y = modular_sqrt(BG_int**3+a*BG_int+b_i,p)
    point_BG = ECPoint(curve, BG_int, y)

    bal, BG2 = get_decryption(-gen, gen*2) # gen*(da+2)
    BG_int2 = int.from_bytes(BG2)
    if (point_BG+gen).x == BG_int2:
        recovered_order[order] = bruteforce(point_BG, gen, order)
    elif (point_BG-gen).x == BG_int2:
        recovered_order[order] = bruteforce(-point_BG, gen,
order)
    else:
        print("something went wrong")

mods, values = [],[]
for i,v in recovered_order.items():
    mods.append(i)
    values.append((v-1)%i)

d_a = crt(mods,values)[0]

key_int = (d_a >> (8*16)) ^ (d_a &
0xffffffffffffffffffffffffffffffff)
key = key_int.to_bytes(16, 'big')
print(AES.new(key, AES.MODE_ECB).decrypt(flag2_enc))
```

ENO{be5t_th1nk_out_0f_th3_curv3}

Solve script

solve.py (./solve.py)

Suggest a correction to this article.

PREVIOUS

ACSC qualifiers 2023 Crypto - SusCipher

NEXT

.inputrc: VI experience in the shell